

Testbarer, dokumentierter Code

Diese Doku haben wir bewusst selbst geschrieben, in eigenen Worten und nicht von der KI generiert. Sie soll zeigen, dass wir verstehen, was wir gebaut haben, und sie soll es einfach machen, das Projekt zu starten und zu testen. Der Code selbst liegt in zwei getrennten Repositories, einem für das Frontend und einem für das Backend.

- **Frontend:** <https://github.com/lorenzo689/vibecoding-frontend>
- **Backend:** <https://github.com/Azockgg/lernapp>

Überblick über den Aufbau

Die App besteht aus zwei Teilen, die klar getrennt sind.

- **Frontend.** Gebaut mit Next.js, React und TypeScript. Es ist das, was die Nutzerin oder der Nutzer im Browser sieht, also Kurse, Dokumente, Zusammenfassungen, Karteikarten, Kalender, Noten und der KI-Assistent.
- **Backend.** Umgesetzt mit Supabase, also einer Postgres-Datenbank mit Migrationen, Sicherheitsregeln und serverseitigen Funktionen. Hier liegen die Daten und die Logik für die KI-Antworten.

Beide Teile sind über eine klar definierte Schnittstelle verbunden. Das Frontend spricht über den Supabase-Client mit der Datenbank. Welche Daten dabei sichtbar sind, entscheiden nicht Filter im Frontend, sondern die Sicherheitsregeln (Row Level Security) in der Datenbank.

Wie das Herzstück funktioniert

Das wichtigste Feature ist der kursbezogene KI-Chat. Damit er sich auf das echte Kursmaterial stützt, läuft im Hintergrund folgender Ablauf.

1. Ein Dokument wird hochgeladen und im Backend verarbeitet.
2. Der Text wird in kleine Abschnitte zerlegt, die sogenannten Chunks.
3. Zu jedem Abschnitt wird ein Vektor berechnet, der seine Bedeutung abbildet.
4. Stellt jemand eine Frage, suchen wir die inhaltlich passendsten Abschnitte über eine Vektorsuche.
5. Diese Abschnitte bekommt die KI als Grundlage für ihre Antwort. Angezeigt wird die Antwort zusammen mit der Quelle zu Folie und Seite.

Dadurch wird die Antwort gezielt auf das eigene Kursmaterial gestützt, statt nur eine allgemeine Antwort wie bei einem freien Chatbot zu erzeugen. Eine absolute Garantie ist das nicht, deshalb zeigen wir immer die Quelle an, damit man die Antwort überprüfen kann.

So startet man das Projekt

Backend und Frontend haben unterschiedliche Node-Versionen. Das Backend braucht die Version aus der Datei `.nvmrc` (aktuell 24.20.0), das Frontend Node 22 oder neuer.

Für die normale lokale Frontend-Entwicklung verbindet sich das Frontend mit dem Supabase-Projekt auf **Staging**, so sieht es die `.env.example` im Frontend vor. Der KI-Chat läuft ausschließlich gegen dieses Projekt, einen Fallback auf den lokalen Supabase-Stack gibt es nicht. Der lokale Supabase-Stack aus dem Backend dient für Datenbank- und Backend-Tests und für die lokale E2E-Suite des Frontends.

1. Backend lokal starten (für Datenbank- und Backend-Tests)

Voraussetzung sind Node in der Version aus der Datei `.nvmrc`, npm und ein laufendes Docker.

```
nvm use
npm ci
npm run db:start
npm run db:status
```

`npm run db:status` zeigt die lokalen Adressen und die Schlüssel an. Geheime Schlüssel wie den Service-Role-Key tragen wir nirgendwo im Frontend ein.

Zum Ausprobieren gibt es zwei vorbereitete **lokale Testkonten**, `anna@example.com` und `ben@example.com`, jeweils mit dem Passwort `password123`. Sie existieren nur in der lokalen Entwicklungsdatenbank und nicht in Staging oder Produktion. Die authentifizierte lokale E2E-Suite des Frontends (`npm run test:e2e:local`) verwendet genau diesen lokalen Stack mit diesen Konten.

2. Frontend starten

Voraussetzung ist Node 22 oder neuer.

```
npm ci
```

Danach die Datei `.env.example` nach `.env.local` kopieren und die öffentlichen Werte des verwendeten Supabase-Projekts (Staging) eintragen.

```
NEXT_PUBLIC_SUPABASE_URL=<project-url>
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=<publishable-key>
AUTH_SITE_URL=http://127.0.0.1:3000
```

Dann den Entwicklungsserver starten.

```
npm run dev
```

Die App ist anschließend unter `http://127.0.0.1:3000` erreichbar. Wichtig ist, dass keine geheimen Schlüssel in das Repository gelangen. Im Frontend werden ausschließlich die öffentlichen Werte verwendet.

So testet man das Projekt

Der Code ist testbar, und die wichtigsten Prüfungen laufen zusätzlich automatisiert über die CI. Von Hand lassen sie sich so starten.

Im Backend prüfen wir die Datenbank und die serverseitigen Funktionen.

```
npm run test:db  
npm run db:lint
```

Im Frontend prüfen wir Tests, Stil und einen vollständigen Build.

```
npm test  
npm run lint  
npm run build
```

Wenn diese Befehle erfolgreich durchlaufen, sind die vorgesehenen automatisierten Prüfungen bestanden. Zusätzlich prüfen wir den vollständigen Ablauf manuell in der Demo.

So läuft die Demo

Für die Vorführung starten wir das Frontend wie oben beschrieben, verbunden mit dem Staging-Projekt, und melden uns mit einem Testkonto auf Staging an. Die lokalen Konten Anna und Ben gibt es dort nicht, sie gelten nur für den lokalen Stack. Dann zeigen wir den durchgehenden Ablauf aus Kurs, Material, Zusammenfassung und Karteikarten sowie den KI-Assistenten, der mit Quellenangabe aus dem Kursmaterial antwortet.

Hinweis zur Dokumentation

Neben dieser Übersicht haben wir die wichtigsten Teile im Code in eigenen Dokumenten festgehalten, zum Beispiel zur Vektorsuche, zum Datei-Speicher, zur Anmeldung und zum Datenmodell. Diese Dokumente liegen in den Repositories im Ordner `docs`. Sie sind von uns geschrieben und helfen uns, den Code auch später noch zu verstehen und sauber zu erweitern.

Im Frontend-Repository liegen zum Beispiel diese Dokumente:

- `docs/authentication.md` zur Anmeldung
- `docs/chat.md` zum KI-Chat
- `docs/e2e.md` zu den End-to-End-Tests
- `docs/public-design-assets.md` zu den öffentlichen Design-Assets

Im Backend-Repository liegen zum Beispiel diese Dokumente:

- `docs/database.md` zum Datenmodell und zu den Beziehungen
- `docs/auth.md` zu Login, Frontend-Anbindung und Row Level Security
- `docs/storage.md` zu privaten Dateien und Storage-Policies
- `docs/rag.md` zu Dokument-Chunks, Embeddings und Vektorsuche
- `docs/summaries.md` zu den Zusammenfassungen
- `docs/flashcard-generation.md` zur Karteikarten-Generierung
- `docs/chat.md` zu Fragen an die Lernmaterialien mit Quellenangaben
- `docs/development.md` und `docs/environments.md` zu Entwicklung, Tests und den Umgebungen (lokal, Staging, Produktion)